

System Design Document: API To-Do List

Este documento técnico descreve a arquitetura, as especificações de design e as estratégias de segurança da **API To-Do List**. O sistema foi concebido para fornecer uma solução de gerenciamento de tarefas, utilizando práticas modernas de desenvolvimento em **Java** e **Spring Boot**.

1. Visão Geral do Projeto

- **Nome do Projeto:** To-Do List.
- **Descrição:** API de Gerenciamento de Tarefas baseada em **Java 17** e **Spring Boot 3.0.11**, focada em prover uma estrutura simplificada para organização de atividades com persistência relacional.
- **Stakeholders:** Usuários finais que necessitam de organização pessoal e desenvolvedores que buscam uma base de API RESTful em Spring Boot.
- **Premissas:**
 - Arquitetura baseada em **monólito**.
 - Persistência inicial utilizando banco de dados **H2**.
 - Autenticação simplificada para fins de desenvolvimento ágil.
- **Resumo de Casos de Uso:**
 - **User Registration:** Criação de conta com persistência de credenciais criptografadas.
 - **Task CRUD:** Fluxos completos de criação, leitura, atualização e listagem de tarefas vinculadas ao usuário autenticado.

2. Requisitos

Requisitos Funcionais

- **Cadastro de Usuário:** O sistema deve permitir o registro de novos usuários com username **Unique**.
- **Gestão de Tarefas:** Criação e listagem de tarefas associadas ao identificador do usuário logado.
- **Validação de Datas:** Bloqueio de criação de tarefas com datas de início (startAt) ou término (endAt) retroativas à data atual.
- **Regra de Negócio de Intervalo:** A data de início deve obrigatoriamente preceder a data de término (requisito de integridade lógica).
- **Atualização Parcial:** Suporte ao **Patch-style update** via método `copyNonNullProperties`, permitindo modificar campos específicos sem sobrescrever o objeto integralmente.
- **Constraint de Título:** O campo title deve possuir uma **Constraint** de validação que limita o conteúdo a, no máximo, 50 caracteres.

Requisitos Não-Funcionais

- **Segurança:** Utilização de **BCrypt** para **Hashing** de senhas e **Basic Auth** para controle de acesso.
- **Performance:** Inclusão do **Spring Boot DevTools** para suporte a **Hot Swap** durante o ciclo de desenvolvimento.

- **Manutenibilidade:** Uso estratégico do **Lombok** no nível de **Model**, especificamente a anotação **@Data**, para automatizar a geração de **Getters**, **Setters** e reduzir o código **Boilerplate**.

3. Arquitetura do Sistema

Tecnologias Utilizadas (Technology Stack)

Categoria	Tecnologia	Versão / Escopo
Language	Java	17
Framework	Spring Boot	3.0.11
Build Tool	Maven	3.x
Database	H2 Database	Runtime Scope
Security Library	at.favre.lib:bcrypt	0.10.2
Utilities	Lombok	Provided Scope

Componentes do Sistema

A organização de pacotes segue o padrão de responsabilidade única, utilizando

Dependency Injection via anotação **@Autowired**:

- **Controller:** Gerencia os **Endpoints** e a lógica de entrada da **API**.
- **Model:** Contém as **JPA Entities** (ex: UserModel, TaskModel) que mapeiam o **Schema** do banco de dados.
- **Repository:** Interfaces que estendem JpaRepository para abstração da persistência.
- **Filter:** Implementa o **Middleware** de segurança para interceptação de requisições.
- **Errors:** Gerenciamento centralizado de exceções através de **@ControllerAdvice**.
- **Utils:** Métodos utilitários para manipulação de **POJOs** e reflexão de propriedades.

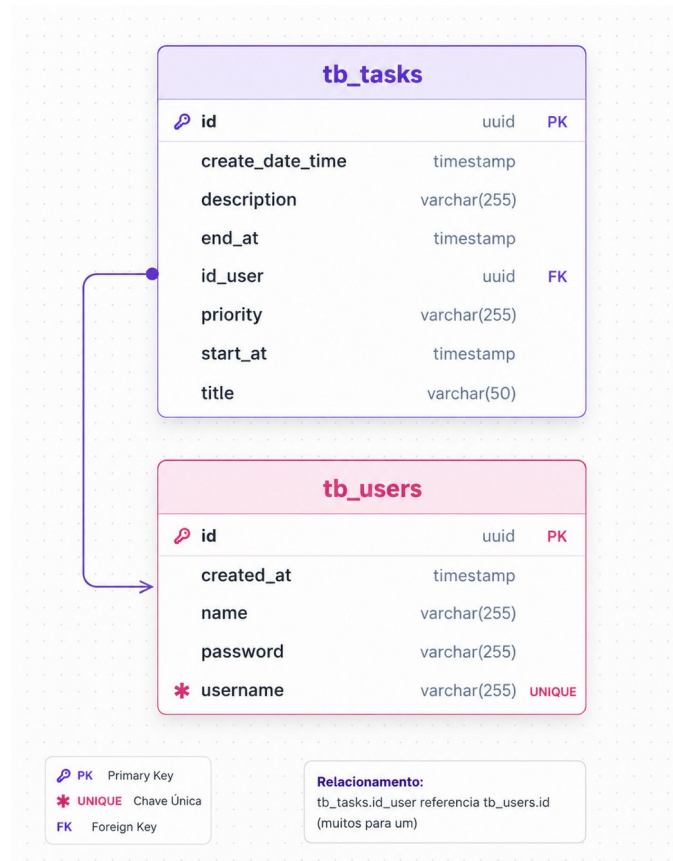
4. Design de Módulos (Módulo Crítico: FilterTaskAuth)

- **Propósito e Fluxo:** O componente FilterTaskAuth estende OncePerRequestFilter para processar a autenticação em cada ciclo de requisição.
- **Lógica Interna:**
 - Interceptação de requisições enviadas ao **Servlet Path** iniciado em /tasks/.
 - Extração do **Header "Authorization"**.
 - Tratamento da **String** para remoção do prefixo **"Basic"** e decodificação do conteúdo em **Base64**.
 - Separação das credenciais (username:password).
 - Validação de segurança: Comparação da senha fornecida com o hash armazenado no banco de dados utilizando a biblioteca **BCrypt**.
- **Saídas:**
 - **Sucesso:** O filtro injeta o idUser no atributo da requisição e invoca o próximo elemento da **FilterChain**.

- **Falha:** Retorno imediato de erro **HTTP 401 (Unauthorized)**.

5. Design do Banco de Dados

Modelagem de Dados (Schema Design)



6. Design da API

Endpoints

Método	Endpoint	Função	Auth Requerida
POST	/users/	Registro de novos usuários	Não
POST	/tasks/	Criação de nova tarefa	Sim (Basic Auth)
GET	/tasks/	Listagem de tarefas do usuário	Sim (Basic Auth)
PUT	/tasks/{id}	Atualização parcial de tarefa	Sim (Basic Auth)

Tratamento de Erros

Implementado através do `ExceptionHandlerController`. O sistema captura `HttpMessageNotReadableException` para tratar erros de **Payload** malformado (como formatos de data inválidos) e retorna mensagens customizadas com status **HTTP 400 (Bad Request)**.

7. Design de Segurança

- **Criptografia:** O `UserController` utiliza o **BCrypt** com um custo de **12 rounds** para o processo de **Hashing**. Este valor foi selecionado para balancear segurança contra ataques de força bruta e custo computacional no servidor.
- **Autorização e Ownership:** No `TaskController`, o sistema realiza uma validação de propriedade rigorosa. Antes de qualquer atualização (PUT), verifica-se se o `idUser` da tarefa no banco coincide com o `idUser` injetado pelo filtro de autenticação. Se `!task.getIdUser().equals(idUser)`, a operação é negada para prevenir acessos não autorizados.

8. Arquitetura de Implantação (Deployment)

O sistema utiliza a estratégia de **Multi-stage Build** em **Docker** para otimizar o tamanho da imagem final:

- **Stage Build:**
 - Ambiente: `ubuntu:latest`.
 - Instalação de ferramentas: **OpenJDK 17 e Maven**.
 - Processo: Execução de `mvn clean install` para gerar o artefato.
- **Stage Runtime:**
 - Imagem base: `openjdk:17-jdk-slim`.
 - Artefato: O arquivo `todolist-1.0.0.jar` é copiado do estágio anterior e renomeado para `app.jar`.
- **Entrypoint:** Comando `java -jar app.jar` expondo a porta **8080**.

9. Estratégia de Testes

Context Testing: A classe `TodolistApplicationTests` utiliza a anotação `@SpringBootTest` para validar se o contexto do **Framework** carrega corretamente, garantindo a integridade da **Dependency Injection** e dos **Beans** de configuração.